CPMSoc × SECSoc Number Theory and Cryptography Workshop

Jeff, Brendan, Vir, Patrick, Peter, Tyson, Teddy and Will

UNSW Competitive Programming and Mathematics Society
&
UNSW Security Society

April 14, 2026

```
    \x90    \x90    \x90
  \x90    \x90    \x90
\x90    \x90    \x90
\x90    \x90    \x90
\x90    \x90    \x90
\x31 \x68 \x59 \x39 \x68    \x83
\x29 \x3e \x3a \x68 \x65    \x83
\x73 \x6f \x68 \x73 \x77    \x6f
  \x77    \x68    \x3c
    \x20    \x73    \x63
    \x20    \x70    \x63
    \x59    \x67    \x90
\x94 \xcd \x80 \x98 \x90    \x90
\x90 \x90 \x90 \x90 \x90    \x90
\x90 \x90 \x90 \x90 \x90    \x90
    \x90    \x90
    \x90    \x90
\x90    \x90    \x90
\x90    \x90    \x90
\x90    \x90    \x90
```

Attendance

Please scan the QR code; please let us know about any dietary requirements (for the pizza!).

Revision

Modular Arithmetic

Definition

We say $a \equiv b \pmod{n}$ when $n \mid (a - b)$. This is equivalent to "a and b yield the same remainder when divided by n".

Modular Arithmetic

Definition

We say $a \equiv b \pmod{n}$ when $n \mid (a - b)$. This is equivalent to "a and b yield the same remainder when divided by n".

Example

$67 \equiv 13 \equiv 4 \equiv -41 \pmod{9}$

See $67 = 13 + 9 \cdot 6$, $13 = 4 + 9 \cdot 1$, $4 = -41 + 9 \cdot 5$

Properties of Modular Arithmetic

Properties of Modular Arithmetic

- $n \equiv 0 \pmod{n}$

Properties of Modular Arithmetic

Properties of Modular Arithmetic

- $n \equiv 0 \pmod{n}$
- If $a \equiv b \pmod{n}$, then $a + k \equiv b + k \pmod{n}$

Properties of Modular Arithmetic

Properties of Modular Arithmetic

- $n \equiv 0 \pmod{n}$
- If $a \equiv b \pmod{n}$, then $a + k \equiv b + k \pmod{n}$
- If $a \equiv b \pmod{n}$, then $ka \equiv kb \pmod{n}$

Properties of Modular Arithmetic

Properties of Modular Arithmetic

- $n \equiv 0 \pmod{n}$
- If $a \equiv b \pmod{n}$, then $a + k \equiv b + k \pmod{n}$
- If $a \equiv b \pmod{n}$, then $ka \equiv kb \pmod{n}$
- If $a \equiv b \pmod{n}$, then $a^k \equiv b^k \pmod{n}$

Properties of Modular Arithmetic

Properties of Modular Arithmetic

- $n \equiv 0 \pmod{n}$
- If $a \equiv b \pmod{n}$, then $a + k \equiv b + k \pmod{n}$
- If $a \equiv b \pmod{n}$, then $ka \equiv kb \pmod{n}$
- If $a \equiv b \pmod{n}$, then $a^k \equiv b^k \pmod{n}$
- If $ka \equiv kb \pmod{n}$, and $\gcd(k, n) = 1$, then $a \equiv b \pmod{n}$.

Properties of Modular Arithmetic

Properties of Modular Arithmetic

- $n \equiv 0 \pmod{n}$
- If $a \equiv b \pmod{n}$, then $a + k \equiv b + k \pmod{n}$
- If $a \equiv b \pmod{n}$, then $ka \equiv kb \pmod{n}$
- If $a \equiv b \pmod{n}$, then $a^k \equiv b^k \pmod{n}$
- If $ka \equiv kb \pmod{n}$, and $\gcd(k, n) = 1$, then $a \equiv b \pmod{n}$.
- If p is prime, then for every a where $p \nmid a$, there exists b such that $ab \equiv 1 \pmod{p}$, but more on this later ...

Useful theorems

Fermat's Little Theorem

If p is prime, then

$$a^p \equiv a \pmod{p}.$$

Wilson's Theorem

If p is prime, then

$$(p-1)! \equiv -1 \pmod{p}$$

Proof of Wilson's Theorem

Solution

Suppose p is prime, then consider the set $S = \{1, 2, 3, ..., p-1\}$. Since p is prime, we know that every number in this set has its own unique multiplicative inverse modulo p.

Proof of Wilson's Theorem

Solution

Suppose p is prime, then consider the set $S = \{1, 2, 3, ..., p-1\}$. Since p is prime, we know that every number in this set has its own unique multiplicative inverse modulo p. Now note that for some integer $1 \leq k \leq p-1$

$$(p-k)^2 = p^2 - 2pk + k^2 \equiv k^2 \pmod{p}$$

so the only elements in S that are their own inverse modulo p are $p-1$ and 1.

Proof of Wilson's Theorem

Solution

Suppose p is prime, then consider the set $S = \{1, 2, 3, ..., p-1\}$. Since p is prime, we know that every number in this set has its own unique multiplicative inverse modulo p. Now note that for some integer $1 \leq k \leq p-1$

$$(p-k)^2 = p^2 - 2pk + k^2 \equiv k^2 \pmod{p}$$

so the only elements in S that are their own inverse modulo p are $p-1$ and 1. Since the set $\{2, 3, ..., p-2\}$ is of even cardinality, we can match up each number with its inverse and so

$$(p-1)! = 1 \cdot 2 \cdot \cdots \cdot (p-2) \cdot (p-1) \equiv (p-1) \equiv -1 \pmod{p}$$

and we are done!

Totient Function

Definition

$\phi(n)$ counts the number of positive integers $k < n$ coprime to n (i.e $\gcd(k, n) = 1$).

Totient Function

Definition

$\phi(n)$ counts the number of positive integers $k < n$ coprime to n (i.e $\gcd(k, n) = 1$).

Product Formula:

$$\phi(n) = n \prod_{p|n} \left(1 - \frac{1}{p}\right) = p_1^{k_1-1}(p_1 - 1)p_2^{k_2-1}(p_2 - 1) \cdots p_r^{k_r-1}(p_r - 1)$$

where $n = p_1^{k_1} p_2^{k_2} \ldots p_r^{k_r}$ is the prime factorisation of n.

Properties of Totient Function

Properties

This formula can be proved using 2 important properties:

- Multiplicative rule:

$$\text{If } \gcd(a, b) = 1, \ \phi(ab) = \phi(a)\phi(b)$$

Properties of Totient Function

Properties

This formula can be proved using 2 important properties:

- Multiplicative rule:

$$\text{If } \gcd(a, b) = 1,\ \phi(ab) = \phi(a)\phi(b)$$

- Value of prime powers:

$$\text{If } p \text{ is prime and } k \geq 1,\ \phi(p^k) = p^k - p^{k-1} = p^{k-1}(p-1) = p^k\left(1 - \frac{1}{p}\right)$$

Euler's Totient Theorem

Euler's Totient Theorem

We can generalise Fermat's Little Theorem with Euler's Totient Theorem, which states, if $\gcd(a, n) = 1$:

$$a^{\phi(n)} \equiv 1 \pmod{n}$$

Euler's Totient Theorem

Euler's Totient Theorem

We can generalise Fermat's Little Theorem with Euler's Totient Theorem, which states, if $\gcd(a, n) = 1$:

$$a^{\phi(n)} \equiv 1 \pmod{n}$$

e.g. $72 = 2^3 \times 3^2$, so $\phi(72) = 24$, so $23^{24} \equiv 1 \pmod{72}$

Chinese Remainder Theorem and Bezout's Identity

Euclidean Algorithm

Algorithm

Suppose we wish to find $\gcd(a, b)$, where $a, b \in \mathbb{Z}$ and $b < a$, then

$$\gcd(a, b) = \gcd(b, r), \text{ where } r = b \pmod{a},$$

and we repeat this process on b, r until the final remainder is 0. Then, the final non-zero remainder is the gcd.

Euclidean Algorithm Example

Example

What is $\gcd(1267, 67)$?

Euclidean Algorithm Example

Example

What is $\gcd(1267, 67)$?

Solution

Using the Euclidean Algorithm, we get that

$$\gcd(1267, 67) = \gcd(67, 61) = \gcd(61, 6) = \gcd(6, 1) = \gcd(1, 0),$$

so $\gcd(1267, 67) = 1$.

Bezout's Identity

Theorem

Let a, b be non-zero integers and $c \in \mathbb{Z}$. Then there exist solutions $x, y \in \mathbb{Z}$ to the equation

$$ax + by = c$$

if and only if $\gcd(a, b) \mid c$.

Bezout's Identity

Theorem

Let a, b be non-zero integers and $c \in \mathbb{Z}$. Then there exist solutions $x, y \in \mathbb{Z}$ to the equation

$$ax + by = c$$

if and only if $\gcd(a, b) \mid c$. We solve these equations using the Euclidean algorithm from before by considering $\gcd(a, b)$ and backtracking. What does this tell us about cancellation in $\mathbb{Z}/b\mathbb{Z}$?

Chinese Remainder Theorem

Theorem

Let $m_1, m_2, \ldots, m_k$ be pairwise coprime positive integers. For any integers $a_1, a_2, \ldots, a_k$, the system

$$x \equiv a_1 \pmod{m_1}, \quad \ldots, \quad x \equiv a_k \pmod{m_k}$$

has a unique solution modulo $M = m_1 m_2 \cdots m_k$.

Semi-67 example

Example

When students are split into groups of 5, there are 2 left over. When split into groups of 9, there are 4 left over. How many students are there, if the answer is less than 100?

Solution

Solution

Let x be the number of students, then we have

$$x \equiv 2 = a_1 \pmod 5, \; x \equiv 4 = a_2 \pmod 9.$$

Solution

Solution

Let x be the number of students, then we have

$$x \equiv 2 = a_1 \pmod 5, \; x \equiv 4 = a_2 \pmod 9.$$

Now, let $M = 45, M_1 = 9, M_2 = 5$ and note that

$$y_1 = 9^{-1} \equiv 4 \pmod 5, \; y_2 = 5^{-1} \equiv 2 \pmod 9.$$

Solution

Solution

Let x be the number of students, then we have

$$x \equiv 2 = a_1 \pmod 5, \; x \equiv 4 = a_2 \pmod 9.$$

Now, let $M = 45, M_1 = 9, M_2 = 5$ and note that

$$y_1 = 9^{-1} \equiv 4 \pmod 5, \; y_2 = 5^{-1} \equiv 2 \pmod 9.$$

Then the solution is given by

$$x \equiv a_1 M_1 y_1 + a_2 M_2 y_2 \equiv 22 \pmod{45},$$

so x is either 22 or 67, and it is easy to verify that both solutions satisfy the conditions.

Quadratic Residues

Quadratic Residues

Definition

Let a and m be integers, with $m \neq 0$. We say that a is a quadratic residue modulo m if there is some integer n so that $n^2 \equiv a \pmod{m}$.

Quadratic Residues

Definition

Let a and m be integers, with $m \neq 0$. We say that a is a quadratic residue modulo m if there is some integer n so that $n^2 \equiv a \pmod{m}$.

Quadratic residues are useful because, under certain moduli, only a small fraction of possible remainders are perfect squares.

Number of Residues

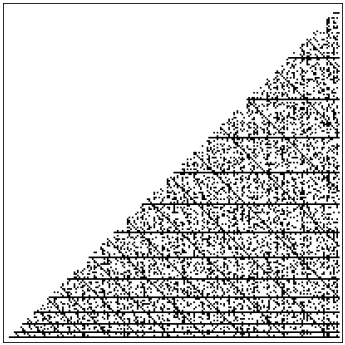


Figure: Shaded cells on column n indicate residues modulo n, note triangular boundary reflecting the $\lfloor n/2 \rfloor$ upper bound.

Number of Residues

The number of quadratic residues modulo n has the upper bound of $\lfloor \frac{n}{2} \rfloor$.

Intuition: Since $x^2 \equiv (n-x)^2 \pmod{n}$, each pair of numbers $\{1, n-1\}, \{2, n-2\}, \ldots$ contribute at most 1 distinct quadratic residue.

Number of Residues

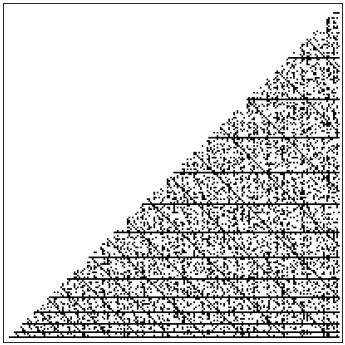


Figure: Shaded cells on column n indicate residues modulo n, note triangular boundary reflecting the $\lfloor n/2 \rfloor$ upper bound.

Number of Residues

The number of quadratic residues modulo n has the upper bound of $\lfloor \frac{n}{2} \rfloor$.

Intuition: Since $x^2 \equiv (n-x)^2 \pmod{n}$, each pair of numbers $\{1, n-1\}, \{2, n-2\}, \ldots$ contribute at most 1 distinct quadratic residue.

Hence, quadratic residues can be used as a filter for various modular arithmetic questions.

Example Problem

Example

Prove that no integer of the form $4k + 3$ can be written as a sum of two squares.

Example Problem

Example

Prove that no integer of the form $4k + 3$ can be written as a sum of two squares.

Solution

Suppose that $4k + 3 = a^2 + b^2$, for some integers a, b and consider the equation modulo 4. Since the quadratic residues of 4 are 0 and 1, we know that $a^2 + b^2 \equiv 0, 1$ or $2 \not\equiv 3 \mod 4$, so no integer of the form exists.

Legendre Symbol

Definition

Let p be an odd prime and let a be an integer.
The Legendre symbol of a with respect to p is defined by

$$\left(\frac{a}{p}\right) = \begin{cases} 0 & \text{if } p \mid a, \\ 1 & \text{if } p \nmid a \text{ and } a \text{ is a quadratic residue modulo } p, \\ -1 & \text{if } p \nmid a \text{ and } a \text{ is a quadratic nonresidue modulo } p. \end{cases}$$

Basic Properties

Some basic properties of the Legendre symbol include:

Properties

Periodicity: if $a \equiv b \pmod{p}$, then

$$\left(\frac{a}{p}\right) = \left(\frac{b}{p}\right).$$

Basic Properties

Some basic properties of the Legendre symbol include:

Properties

Periodicity: if $a \equiv b \pmod{p}$, then

$$\left(\frac{a}{p}\right) = \left(\frac{b}{p}\right).$$

Multiplicativity:

$$\left(\frac{ab}{p}\right) = \left(\frac{a}{p}\right)\left(\frac{b}{p}\right).$$

The proofs for these aren't too hard from first principles, so try to prove them for yourself!

Quadratic Reciprocity

- So far, the only way we know for evaluating $\left(\frac{a}{p}\right)$ is by manually finding and checking quadratic residues modulo p.

Quadratic Reciprocity

- So far, the only way we know for evaluating $\left(\frac{a}{p}\right)$ is by manually finding and checking quadratic residues modulo p.
- This quickly becomes very difficult and tedious for large p.

Quadratic Reciprocity

- So far, the only way we know for evaluating $\left(\frac{a}{p}\right)$ is by manually finding and checking quadratic residues modulo p.
- This quickly becomes very difficult and tedious for large p.
- For bigger primes, we introduce the law of Quadratic Reciprocity.

The Law of Quadratic Reciprocity

Theorem

Let p and q be distinct odd primes. The statement of the main theorem is:

$$\left(\frac{p}{q}\right)\left(\frac{q}{p}\right) = (-1)^{(p-1)(q-1)/4},$$

$$\left(\frac{-1}{p}\right) = (-1)^{(p-1)/2},$$

$$\left(\frac{2}{p}\right) = (-1)^{(p^2-1)/8}.$$

Note: Most definitions will only include the first statement, known as the "main theorem", while the latter two are generally referred to as the first and second supplement to the law.

Application of Quadratic Reciprocity

Example

Does the congruence $x^2 + 6x + 15 \equiv 0 \pmod{41}$ have a solution?

Application of Quadratic Reciprocity

Example

Does the congruence $x^2 + 6x + 15 \equiv 0 \pmod{41}$ have a solution?

Solution

Re-write the congruence by $(x+3)^2 \equiv 35 \pmod{41}$. A solution exists if and only if 35 is a quadratic residue mod 41, i.e. $\left(\frac{35}{41}\right) = 1$.

Application of Quadratic Reciprocity

Example

Does the congruence $x^2 + 6x + 15 \equiv 0 \pmod{41}$ have a solution?

Solution

Re-write the congruence by $(x+3)^2 \equiv 35 \pmod{41}$. A solution exists if and only if 35 is a quadratic residue mod 41, i.e. $\left(\frac{35}{41}\right) = 1$.
We can compute this using multiplicativity and quadratic reciprocity:

$$\left(\frac{35}{41}\right) = \left(\frac{5}{41}\right)\left(\frac{7}{41}\right) = \left(\frac{41}{5}\right)\left(\frac{41}{7}\right) = \left(\frac{1}{5}\right)\left(\frac{-1}{7}\right) = -1.$$

Hence the congruence **has no solution**.

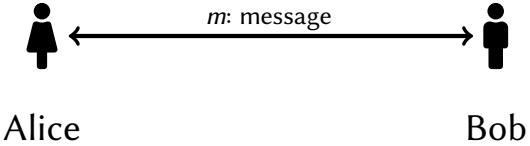
Cryptography

Principles of Cryptography

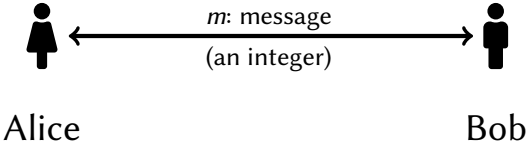
Alice

Bob

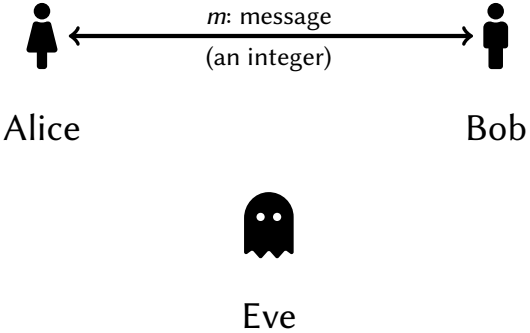
Principles of Cryptography



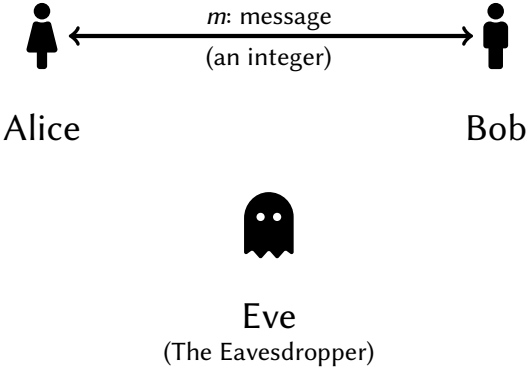
Principles of Cryptography



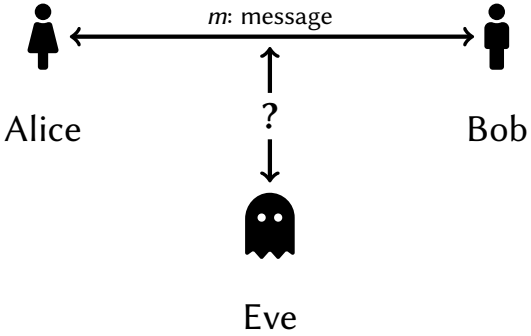
Principles of Cryptography



Principles of Cryptography



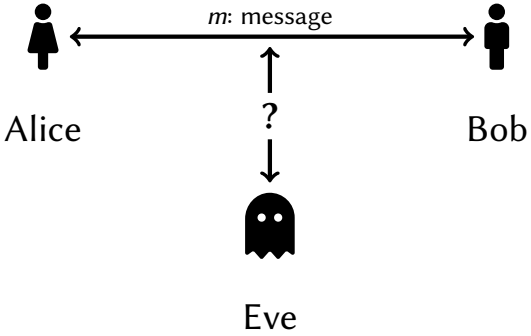
Aims of Cryptography



As Alice...

- Can Eve read my message?
- Can Eve change my message?
- Am I even talking to Bob?

Aims of Cryptography



As Alice...

- **Can Eve read my message?**
- Can Eve change my message?
- Am I even talking to Bob?

RSA

RSA Basics

RSA (Rivest-Shamir-Adleman, not randomly-secure-algorithm) leverages the factorization problem

RSA Basics

RSA (Rivest-Shamir-Adleman, not randomly-secure-algorithm) leverages the factorization problem

- Factorization is a difficult problem

RSA Basics

RSA (Rivest-Shamir-Adleman, not randomly-secure-algorithm) leverages the factorization problem

- Factorization is a difficult problem
- There is no efficient algorithm to factor a really big number

RSA Basics

RSA (Rivest-Shamir-Adleman, not randomly-secure-algorithm) leverages the factorization problem

- Factorization is a difficult problem
- There is no efficient algorithm to factor a really big number
- It is computationally expensive to find the factors of product of two primes

RSA Algorithm - Setup

[Algorithm] - Key Generation

RSA Algorithm - Setup

[Algorithm] - Key Generation

0 Generate primes p, q, usually 1024 bits each

RSA Algorithm - Setup

[Algorithm] - Key Generation

0 Generate primes p, q, usually 1024 bits each

1 $N = p \cdot q$, so N is 2048 bits

RSA Algorithm - Setup

[Algorithm] - Key Generation

0. Generate primes p, q, usually 1024 bits each
1. $N = p \cdot q$, so N is 2048 bits
2. Let $e = 3$ or 65537

RSA Algorithm - Setup

[Algorithm] - Key Generation

0 Generate primes p, q, usually 1024 bits each

1 $N = p \cdot q$, so N is 2048 bits

2 Let $e = 3$ or 65537

3 Compute $d = e^{-1} \pmod{\phi(N)}$

RSA Algorithm - Setup

[Algorithm] - Key Generation

0 Generate primes p, q, usually 1024 bits each

1 $N = p \cdot q$, so N is 2048 bits

2 Let $e = 3$ or 65537

3 Compute $d = e^{-1} \pmod{\phi(N)}$

4 Recall $\phi(N) = \phi(p \cdot q) = (p-1)(q-1)$

RSA Algorithm - Setup

[Algorithm] - Key Generation

0. Generate primes p, q, usually 1024 bits each
1. $N = p \cdot q$, so N is 2048 bits
2. Let $e = 3$ or 65537
3. Compute $d = e^{-1} \pmod{\phi(N)}$
4. Recall $\phi(N) = \phi(p \cdot q) = (p-1)(q-1)$

Public key: (N, e)
Private key: (d)

RSA Algorithm - Setup

[Algorithm] - Key Generation

0. Generate primes p, q, usually 1024 bits each
1. $N = p \cdot q$, so N is 2048 bits
2. Let $e = 3$ or 65537
3. Compute $d = e^{-1} \pmod{\phi(N)}$
4. Recall $\phi(N) = \phi(p \cdot q) = (p-1)(q-1)$

Public key: (N, e)
Private key: (d)

- Notice - u must factor N to compute private key

RSA Algorithm

[Algorithm] - Key Generation

0. Generate primes p, q, usually 1024 bits each
1. $N = p \cdot q$, so N is 2048 bits
2. Let $e = 3$ or 65537
3. Compute $d = e^{-1} \pmod{\phi(N)}$
4. Recall $\phi(N) = \phi(p \cdot q) = (p-1)(q-1)$

Public key: (N, e)
Private key: (d)

[Algorithm] - Encryption

RSA Algorithm

[Algorithm] - Key Generation

0. Generate primes p, q, usually 1024 bits each
1. $N = p \cdot q$, so N is 2048 bits
2. Let $e = 3$ or 65537
3. Compute $d = e^{-1} \pmod{\phi(N)}$
4. Recall $\phi(N) = \phi(p \cdot q) = (p-1)(q-1)$

Public key: (N, e)
Private key: (d)

[Algorithm] - Encryption

0. Let m be the message to encrypt, and c be the ciphertext

RSA Algorithm

[Algorithm] - Key Generation

0. Generate primes p, q, usually 1024 bits each
1. $N = p \cdot q$, so N is 2048 bits
2. Let $e = 3$ or 65537
3. Compute $d = e^{-1} \pmod{\phi(N)}$
4. Recall $\phi(N) = \phi(p \cdot q) = (p-1)(q-1)$

Public key: (N, e)
Private key: (d)

[Algorithm] - Encryption

0. Let m be the message to encrypt, and c be the ciphertext
1. $c = m^e \pmod{N}$

RSA Algorithm

[Algorithm] - Key Generation

0. Generate primes p, q, usually 1024 bits each
1. $N = p \cdot q$, so N is 2048 bits
2. Let $e = 3$ or 65537
3. Compute $d = e^{-1} \pmod{\phi(N)}$
4. Recall $\phi(N) = \phi(p \cdot q) = (p-1)(q-1)$

Public key: (N, e)
Private key: (d)

[Algorithm] - Encryption

0. Let m be the message to encrypt, and c be the ciphertext
1. $c = m^e \pmod{N}$

[Algorithm] - Decryption

RSA Algorithm

[Algorithm] - Key Generation

0. Generate primes p, q, usually 1024 bits each
1. $N = p \cdot q$, so N is 2048 bits
2. Let $e = 3$ or 65537
3. Compute $d = e^{-1} \pmod{\phi(N)}$
4. Recall $\phi(N) = \phi(p \cdot q) = (p-1)(q-1)$

Public key: (N, e)
Private key: (d)

[Algorithm] - Encryption

0. Let m be the message to encrypt, and c be the ciphertext
1. $c = m^e \pmod{N}$

[Algorithm] - Decryption

0. $m = c^d \pmod{N}$

RSA Algorithm

[Algorithm] - Key Generation

0. Generate primes p, q, usually 1024 bits each
1. $N = p \cdot q$, so N is 2048 bits
2. Let $e = 3$ or 65537
3. Compute $d = e^{-1} \pmod{\phi(N)}$
4. Recall $\phi(N) = \phi(p \cdot q) = (p-1)(q-1)$

Public key: (N, e)
Private key: (d)

[Algorithm] - Encryption

0. Let m be the message to encrypt, and c be the ciphertext
1. $c = m^e \pmod{N}$

[Algorithm] - Decryption

0. $m = c^d \pmod{N}$
1. (c deez nuts)

RSA Proof

Proof

What is the meths behind this?

RSA Proof

Proof

What is the meths behind this?

- Recall $a^{\phi(n)} \equiv 1 \pmod{n}$ if $\gcd(a, n) = 1$

RSA Proof

Proof

What is the meths behind this?

- Recall $a^{\phi(n)} \equiv 1 \pmod{n}$ if $\gcd(a, n) = 1$
- Since $d = e^{-1} \pmod{\phi(N)}$, $e \cdot d \equiv 1 \pmod{\phi(N)}$

RSA Proof

Proof

What is the meths behind this?

- Recall $a^{\phi(n)} \equiv 1 \pmod{n}$ if $\gcd(a, n) = 1$
- Since $d = e^{-1} \pmod{\phi(N)}$, $e \cdot d \equiv 1 \pmod{\phi(N)}$
- Thus, $e \cdot d = 1 + k \cdot \phi(N)$

RSA Proof

Proof

As $c = m^e \pmod{N}$,

$$c^d \pmod{N} = (m^e)^d \pmod{N}$$

RSA Proof

Proof

As $c = m^e \pmod{N}$,

$$\begin{aligned} c^d \pmod{N} &= (m^e)^d \pmod{N} \\ &= m^{e \cdot d} \pmod{N} \end{aligned}$$

RSA Proof

Proof

As $c = m^e \pmod{N}$,

$$\begin{aligned} c^d \pmod{N} &= (m^e)^d \pmod{N} \\ &= m^{e \cdot d} \pmod{N} \\ &= m^{1+k \cdot \phi(N)} \pmod{N} \end{aligned}$$

RSA Proof

Proof

As $c = m^e \pmod{N}$,

$$\begin{aligned} c^d \pmod{N} &= (m^e)^d \pmod{N} \\ &= m^{e \cdot d} \pmod{N} \\ &= m^{1+k \cdot \phi(N)} \pmod{N} \\ &= m \cdot m^{k \cdot \phi(N)} \pmod{N} \end{aligned}$$

RSA Proof

Proof

As $c = m^e \pmod{N}$,

$$\begin{aligned} c^d \pmod{N} &= (m^e)^d \pmod{N} \\ &= m^{e \cdot d} \pmod{N} \\ &= m^{1+k \cdot \phi(N)} \pmod{N} \\ &= m \cdot m^{k \cdot \phi(N)} \pmod{N} \\ &= m \pmod{N} \end{aligned}$$

DH

DH Basics

DH (Diffie-Hellman) Key Exchange relies on the **Discrete Log Problem**

DH Basics

DH (Diffie-Hellman) Key Exchange relies on the **Discrete Log Problem**

- Let $g^x = h \pmod{p}$

DH Basics

DH (Diffie-Hellman) Key Exchange relies on the **Discrete Log Problem**

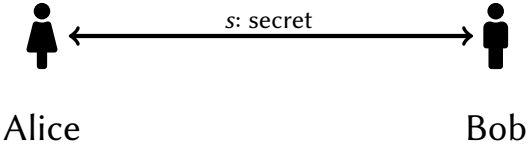
- Let $g^x = h \pmod{p}$
- Find x, given g, h, p

DH Basics

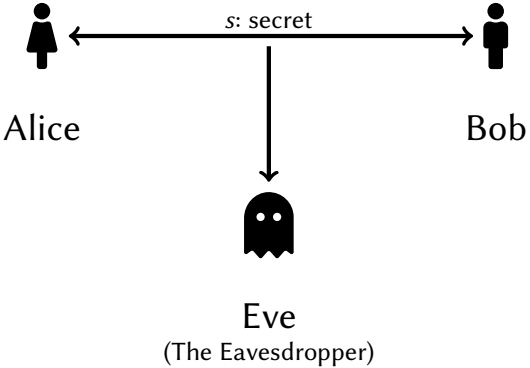
DH (Diffie-Hellman) Key Exchange relies on the **Discrete Log Problem**

- Let $g^x = h \pmod{p}$
- Find x, given g, h, p
- Same as RSA factorization problem - computationally expensive, no efficient known algorithm

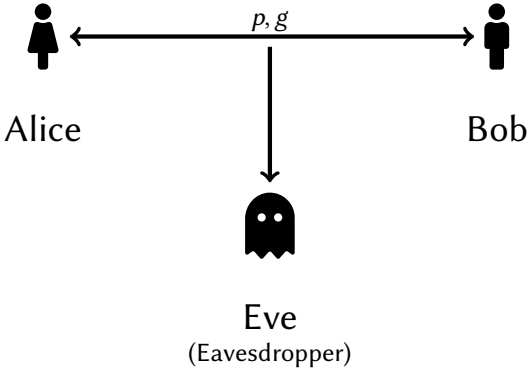
DH Scenario



DH Scenario



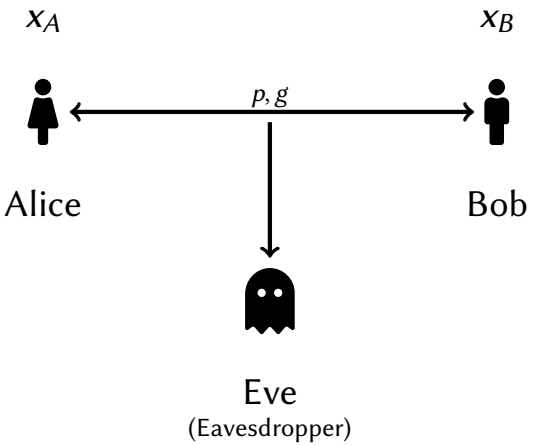
DH Algorithm



Algorithm

- Decide on a prime p and base g

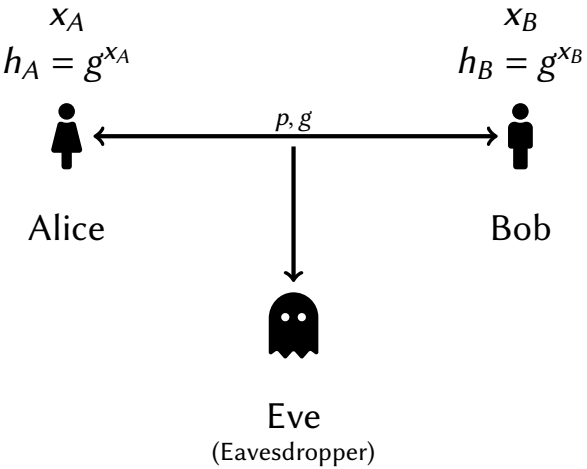
DH Algorithm



Algorithm

- Decide on a prime p and base g
- Alice picks x_A, Bob picks x_B

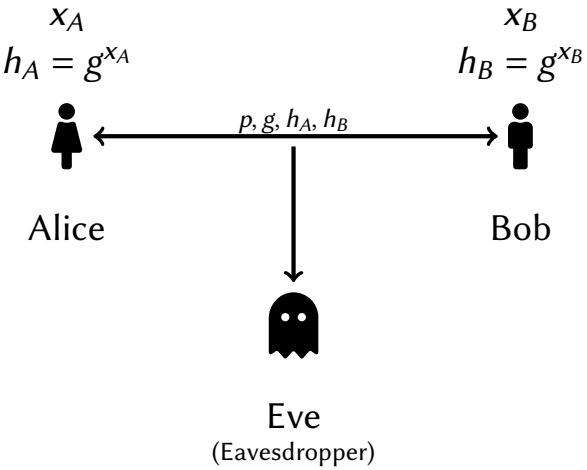
DH Algorithm



Algorithm

- Decide on a prime p and base g
- Alice picks x_A, Bob picks x_B
- Compute $h_A = g^{x_A}$, $h_B = g^{x_B}$

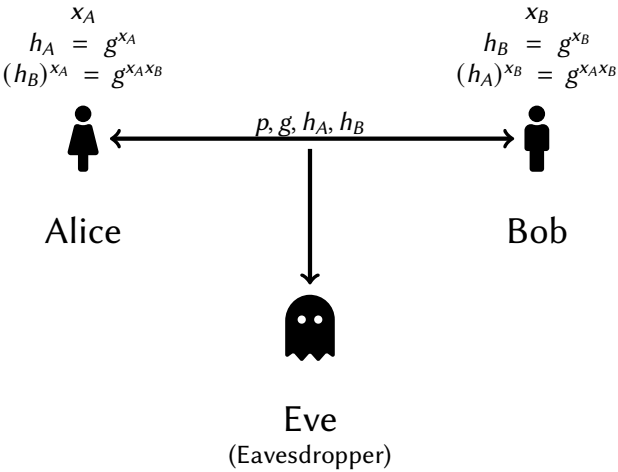
DH Algorithm



Algorithm

- Decide on a prime p and base g
- Alice picks x_A, Bob picks x_B
- Compute $h_A = g^{x_A}$, $h_B = g^{x_B}$
- Exchange h_A, h_B

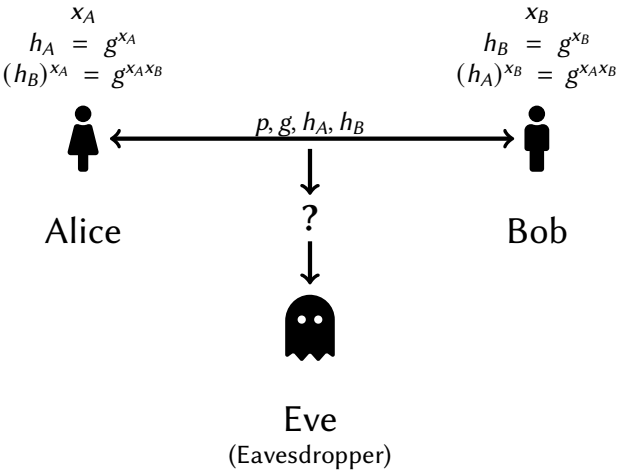
DH Algorithm



Algorithm

- Decide on a prime p and base g
- Alice picks x_A, Bob picks x_B
- Compute $h_A = g^{x_A}$, $h_B = g^{x_B}$
- Exchange h_A, h_B
- Compute shared secret $s = g^{x_A x_B}$

DH Algorithm



Algorithm

- Decide on a prime p and base g
- Alice picks x_A, Bob picks x_B
- Compute $h_A = g^{x_A}$, $h_B = g^{x_B}$
- Exchange h_A, h_B
- Compute shared secret $s = g^{x_A x_B}$

- Eve sees p, g, h_A, h_B but not x_A, x_B

Attacks

Small M

Attack

- If we know that message m is small, such that $m^e < N$

Small M

Attack

- If we know that message m is small, such that $m^e < N$
- $c = m^e \pmod{N} = m^e$

Small M

Attack

- If we know that message m is small, such that $m^e < N$
- $c = m^e \pmod{N} = m^e$
- We can recover $m = \sqrt[e]{c}$ easily

Small M

Attack

- If we know that message m is small, such that $m^e < N$
- $c = m^e \pmod{N} = m^e$
- We can recover $m = \sqrt[e]{c}$ easily

How is this different from discrete log?

Small M

Attack

- If we know that message m is small, such that $m^e < N$
- $c = m^e \pmod{N} = m^e$
- We can recover $m = \sqrt[e]{c}$ easily

How is this different from discrete log?

Small M

Attack

- If we know that message m is small, such that $m^e < N$
- $c = m^e \pmod{N} = m^e$
- We can recover $m = \sqrt[e]{c}$ easily

How is this different from discrete log?

- When $m^e >> N$, $m^e \pmod{N}$ can equal $m^e + N, m^e + 2N, ..., m^e + kN$

Small M

Attack

- If we know that message m is small, such that $m^e < N$
- $c = m^e \pmod{N} = m^e$
- We can recover $m = \sqrt[e]{c}$ easily

How is this different from discrete log?

- When $m^e >> N$, $m^e \pmod{N}$ can equal $m^e + N, m^e + 2N, ..., m^e + kN$
- k can become very big, so we have to take the root multiple times

Small M

Attack

- If we know that message m is small, such that $m^e < N$
- $c = m^e \pmod{N} = m^e$
- We can recover $m = \sqrt[e]{c}$ easily

How is this different from discrete log?

- When $m^e >> N$, $m^e \pmod{N}$ can equal $m^e + N, m^e + 2N, ..., m^e + kN$
- k can become very big, so we have to take the root multiple times
- This is a reason why we using *padding* before encrypting a message

Hastad's Attack

Broadcast Encryption

Hastad's Attack

Broadcast Encryption

- We want to send the same message m to 3 recipients with public keys N_1, N_2, N_3

Hastad's Attack

Broadcast Encryption

- We want to send the same message m to 3 recipients with public keys N_1, N_2, N_3
- Let $c_1 = m^e \pmod{N_1}, c_2 = m^e \pmod{N_2}, c_3 = m^e \pmod{N_3}$

Hastad's Attack

Broadcast Encryption

- We want to send the same message m to 3 recipients with public keys N_1, N_2, N_3
- Let $c_1 = m^e \pmod{N_1}$, $c_2 = m^e \pmod{N_2}$, $c_3 = m^e \pmod{N_3}$
- We can broadcast $[c_1, c_2, c_3]$, with the hope that only the intended recipient can decrypt their ciphertext

Hastad's Attack

Broadcast Encryption

- We want to send the same message m to 3 recipients with public keys N_1, N_2, N_3
- Let $c_1 = m^e \pmod{N_1}$, $c_2 = m^e \pmod{N_2}$, $c_3 = m^e \pmod{N_3}$
- We can broadcast $[c_1, c_2, c_3]$, with the hope that only the intended recipient can decrypt their ciphertext

Attack

Hastad's Attack

Broadcast Encryption

- We want to send the same message m to 3 recipients with public keys N_1, N_2, N_3
- Let $c_1 = m^e \pmod{N_1}$, $c_2 = m^e \pmod{N_2}$, $c_3 = m^e \pmod{N_3}$
- We can broadcast $[c_1, c_2, c_3]$, with the hope that only the intended recipient can decrypt their ciphertext

Attack

- If $e = 3$, as it commonly is...

Hastad's Attack

Broadcast Encryption

- We want to send the same message m to 3 recipients with public keys N_1, N_2, N_3
- Let $c_1 = m^e \pmod{N_1}$, $c_2 = m^e \pmod{N_2}$, $c_3 = m^e \pmod{N_3}$
- We can broadcast $[c_1, c_2, c_3]$, with the hope that only the intended recipient can decrypt their ciphertext

Attack

- If $e = 3$, as it commonly is...
- Find $x = m^3 \pmod{N_1 N_2 N_3}$

Hastad's Attack

Broadcast Encryption

- We want to send the same message m to 3 recipients with public keys N_1, N_2, N_3
- Let $c_1 = m^e \pmod{N_1}$, $c_2 = m^e \pmod{N_2}$, $c_3 = m^e \pmod{N_3}$
- We can broadcast $[c_1, c_2, c_3]$, with the hope that only the intended recipient can decrypt their ciphertext

Attack

- If $e = 3$, as it commonly is...
- Find $x = m^3 \pmod{N_1 N_2 N_3}$
- Evaluate $m = \sqrt[3]{x}$

Hastad's Attack

Attack

- If $e = 3$, as it commonly is...
- Find $x = m^3 \pmod{N_1 N_2 N_3}$
- Evaluate $m = \sqrt[3]{x}$

Proof

- We are given:

$$c_1 = m^3 \pmod{N_1}$$
$$c_2 = m^3 \pmod{N_2}$$
$$c_3 = m^3 \pmod{N_3}$$

Hastad's Attack

Attack

- If $e = 3$, as it commonly is...
- Find $x = m^3 \pmod{N_1 N_2 N_3}$
- Evaluate $m = \sqrt[3]{x}$

Proof

- We are given:

$$c_1 = m^3 \pmod{N_1}$$
$$c_2 = m^3 \pmod{N_2}$$
$$c_3 = m^3 \pmod{N_3}$$

- Without loss of generality, if $gcd(N_1, N_2) \neq 1$, then we can use Euclidean algorithm to factorise N_1, N_2 and win

Hastad's Attack

Attack

- If $e = 3$, as it commonly is...
- Find $x = m^3 \pmod{N_1 N_2 N_3}$
- Evaluate $m = \sqrt[3]{x}$

Proof

- We are given:

$$c_1 = m^3 \pmod{N_1}$$
$$c_2 = m^3 \pmod{N_2}$$
$$c_3 = m^3 \pmod{N_3}$$

- Without loss of generality, if $gcd(N_1, N_2) \neq 1$, then we can use Euclidean algorithm to factorise N_1, N_2 and win
- Thus, we can use Chinese Remainder Theorem to find a unique $x = m^3 \pmod{N_1 N_2 N_3}$

Hastad's Attack

Attack

- If $e = 3$, as it commonly is...
- Find $x = m^3 \pmod{N_1 N_2 N_3}$
- Evaluate $m = \sqrt[3]{x}$

Proof

- If $m < N_1, N_2, N_3$, then $m^3 < N_1 \cdot N_2 \cdot N_3$

Hastad's Attack

Attack

- If $e = 3$, as it commonly is...
- Find $x = m^3 \pmod{N_1 N_2 N_3}$
- Evaluate $m = \sqrt[3]{x}$

Proof

- If $m < N_1, N_2, N_3$, then $m^3 < N_1 \cdot N_2 \cdot N_3$
- Recall that $x = m^3 \mod N_1 N_2 N_3$ in this scenario

Hastad's Attack

Attack

- If $e = 3$, as it commonly is...
- Find $x = m^3 \pmod{N_1 N_2 N_3}$
- Evaluate $m = \sqrt[3]{x}$

Proof

- If $m < N_1, N_2, N_3$, then $m^3 < N_1 \cdot N_2 \cdot N_3$
- Recall that $x = m^3 \mod N_1 N_2 N_3$ in this scenario
- Thus, $x = m^3$, so we can use the small message attack from before!

Hastad's Attack

Attack

- If $e = 3$, as it commonly is...
- Find $x = m^3 \pmod{N_1 N_2 N_3}$
- Evaluate $m = \sqrt[3]{x}$

Proof

- If $m < N_1, N_2, N_3$, then $m^3 < N_1 \cdot N_2 \cdot N_3$
- Recall that $x = m^3 \mod N_1 N_2 N_3$ in this scenario
- Thus, $x = m^3$, so we can use the small message attack from before!

- Note that as long as $m^e \approx N_1 N_2 \ldots N_k$, we can carry out this attack

Hastad's Attack

Attack

- If $e = 3$, as it commonly is...
- Find $x = m^3 \pmod{N_1 N_2 N_3}$
- Evaluate $m = \sqrt[3]{x}$

Proof

- If $m < N_1, N_2, N_3$, then $m^3 < N_1 \cdot N_2 \cdot N_3$
- Recall that $x = m^3 \mod N_1 N_2 N_3$ in this scenario
- Thus, $x = m^3$, so we can use the small message attack from before!

- Note that as long as $m^e \approx N_1 N_2 \ldots N_k$, we can carry out this attack
- If $e = k$, then we **always** have this attack

Hastad's Attack

Attack

- If $e = 3$, as it commonly is...
- Find $x = m^3 \pmod{N_1 N_2 N_3}$
- Evaluate $m = \sqrt[3]{x}$

Proof

- If $m < N_1, N_2, N_3$, then $m^3 < N_1 \cdot N_2 \cdot N_3$
- Recall that $x = m^3 \mod N_1 N_2 N_3$ in this scenario
- Thus, $x = m^3$, so we can use the small message attack from before!

- Note that as long as $m^e \approx N_1 N_2 \ldots N_k$, we can carry out this attack
- If $e = k$, then we **always** have this attack
- Again, if we use randomised padding for every message, this attack is less effective

Franklin-Reiter Attack

Attack

Franklin-Reiter Attack

Attack

- Let Alice have public keys (N, e)

Franklin-Reiter Attack

Attack

- Let Alice have public keys (N, e)
- Let Bob encrypt 2 messages m_1, m_2 to send c_1, c_2 to Alice

Franklin-Reiter Attack

Attack

- Let Alice have public keys (N, e)
- Let Bob encrypt 2 messages m_1, m_2 to send c_1, c_2 to Alice
- If we know that m_1 and m_2 are related, i.e. $m_2 = f(m_1)$ for some polynomial f with coefficients in $\mathbb{Z}_N$, we can recover the messages

Franklin-Reiter Attack

Attack

- Let Alice have public keys (N, e)
- Let Bob encrypt 2 messages m_1, m_2 to send c_1, c_2 to Alice
- If we know that m_1 and m_2 are related, i.e. $m_2 = f(m_1)$ for some polynomial f with coefficients in $\mathbb{Z}_N$, we can recover the messages
- Let $g_1(x) = x^e - c_1$ and $g_2(x) = f(x)^e - c_2$

Franklin-Reiter Attack

Notes

- Alice have public keys (N, e)
- $c_1 = m_1^e \pmod{N}$
- $c_2 = m_2^e \pmod{N}$
- $m_2 = f(m_1)$
- $g_1(x) = x^e - c_1$
- $g_2(x) = f(x)^e - c_2$

Attack

- Note:

$$\begin{aligned} g_1(m_1) &= m_1^e - c_1 \\ &= 0 \\ g_2(m_1) &= f(m_1)^e - c_2 \\ &= m_2^e - c_2 \\ &= 0 \end{aligned}$$

Franklin-Reiter Attack

Notes

- Alice have public keys (N, e)
- $c_1 = m_1^e \pmod{N}$
- $c_2 = m_2^e \pmod{N}$
- $m_2 = f(m_1)$
- $g_1(x) = x^e - c_1$
- $g_2(x) = f(x)^e - c_2$

Attack

- Note:

$$\begin{aligned} g_1(m_1) &= m_1^e - c_1 \\ &= 0 \\ g_2(m_1) &= f(m_1)^e - c_2 \\ &= m_2^e - c_2 \\ &= 0 \end{aligned}$$

- So $(x - m_1)$ is a root to both polynomials

Franklin-Reiter Attack

Notes

- Alice have public keys (N, e)
- $c_1 = m_1^e \pmod{N}$
- $c_2 = m_2^e \pmod{N}$
- $m_2 = f(m_1)$
- $g_1(x) = x^e - c_1$
- $g_2(x) = f(x)^e - c_2$

Attack

- Note:

$$\begin{aligned} g_1(m_1) &= m_1^e - c_1 \\ &= 0 \\ g_2(m_1) &= f(m_1)^e - c_2 \\ &= m_2^e - c_2 \\ &= 0 \end{aligned}$$

- So $(x - m_1)$ is a root to both polynomials
- We can evaluate the **polynomial gcd** of $g_1(x), g_2(x)$ to get $(x - m_1)$ and recover m_1

Franklin-Reiter Attack

Notes

- Alice have public keys (N, e)
- $c_1 = m_1^e \pmod{N}$
- $c_2 = m_2^e \pmod{N}$
- $\boldsymbol{m_2 = f(m_1)} \leftarrow$
- $g_1(x) = x^e - c_1$
- $g_2(x) = f(x)^e - c_2$

Attack

- Note:

$$\begin{aligned} g_1(m_1) &= m_1^e - c_1 \\ &= 0 \\ g_2(m_1) &= f(m_1)^e - c_2 \\ &= m_2^e - c_2 \\ &= 0 \end{aligned}$$

- So $(x - m_1)$ is a root to both polynomials
- We can evaluate the **polynomial gcd** of $g_1(x), g_2(x)$ to get $(x - m_1)$ and recover m_1

Other Cool Attacks

Some other kewl hax you can do (exercise left for the reader):

Other Cool Attacks

Some other kewl hax you can do (exercise left for the reader):

- Weiner's Attack (small d attack)

Other Cool Attacks

Some other kewl hax you can do (exercise left for the reader):

- Weiner's Attack (small d attack)
- Coppersmith's Attack

Other Cool Attacks

Some other kewl hax you can do (exercise left for the reader):

- Weiner's Attack (small d attack)
- Coppersmith's Attack
- Meet in the Middle Algorithm

Other Cool Attacks

Some other kewl hax you can do (exercise left for the reader):

- Weiner's Attack (small d attack)
- Coppersmith's Attack
- Meet in the Middle Algorithm
- a quantum computer

Appendix

Euler's Criterion

Theorem

If p is an odd prime and a is an integer not divisible by p, then

$$\left(\frac{a}{p}\right) \equiv a^{\frac{p-1}{2}} \mod p$$

Discrete Logarithm

Definition

Suppose we have a congruence equation in the form

$$x^{\beta} \equiv a \mod n,$$

then the discrete logarithm of a modulo n with base x is the value of β, which we often denote

$$\beta = \log_x(a).$$

Discrete Logarithm

Definition

Suppose we have a congruence equation in the form

$$x^{\beta} \equiv a \mod n,$$

then the discrete logarithm of a modulo n with base x is the value of β, which we often denote

$$\beta = \log_x(a).$$

We can often think of the discrete logarithm as similar to the normal logarithm function we have learned before; x to what power gives us a modulo n? It also shares similar properties as the normal log function.

Application of discrete logarithm

Example

Solve the congruence equation

$$x^{65} \equiv 6 \mod 7.$$

Application of discrete logarithm

Example

Solve the congruence equation

$$x^{65} \equiv 6 \mod 7.$$

Solution: If we take $\log_3$ on both sides then, we get $65 \log_3(x) \equiv \log_3(6) \mod \varphi(7)$ and simplifying it we get $5 \log_3(x) \equiv 3 \mod 6$ as $3^3 \equiv 6 \mod 7$. Now $5^{-1} \equiv 5 \mod 6$, so we get $\log_3(x) \equiv 3 \mod 6$, so $x \equiv 6 \mod 7$.

Application of discrete logarithm

Example

Solve the congruence equation

$$x^{65} \equiv 6 \mod 7.$$

Solution: If we take $\log_3$ on both sides then, we get $65 \log_3(x) \equiv \log_3(6) \mod \varphi(7)$ and simplifying it we get $5 \log_3(x) \equiv 3 \mod 6$ as $3^3 \equiv 6 \mod 7$. Now $5^{-1} \equiv 5 \mod 6$, so we get $\log_3(x) \equiv 3 \mod 6$, so $x \equiv 6 \mod 7$.
Notice that we switched the modulo when we apply the discrete logarithm? Why?

Application of discrete logarithm

Example

Diffie-Hellman: Two people $1, 2$ establish a secret key K which is n bits long.

- Choose large prime p, longer than n bit

Application of discrete logarithm

Example

Diffie-Hellman: Two people 1, 2 establish a secret key K which is n bits long.

- Choose large prime p, longer than n bit
- Choose g such that $g^i \equiv j \pmod{p}$ for some i and all $j \leq p$

Application of discrete logarithm

Example

Diffie-Hellman: Two people 1, 2 establish a secret key K which is n bits long.

- Choose large prime p, longer than n bit
- Choose g such that $g^i \equiv j \pmod{p}$ for some i and all $j \leq p$
- 1 computes $P_1 = g^{i_1} \pmod{n}$ and 2 computes $P_2 = g^{i_2} \pmod{n}$ where i_1, i_2 are their respective SECRETS

Application of discrete logarithm

Example

Diffie-Hellman: Two people 1, 2 establish a secret key K which is n bits long.

- Choose large prime p, longer than n bit
- Choose g such that $g^i \equiv j \pmod{p}$ for some i and all $j \leq p$
- 1 computes $P_1 = g^{i_1} \pmod{n}$ and 2 computes $P_2 = g^{i_2} \pmod{n}$ where i_1, i_2 are their respective SECRETS
- They exchange P_1 and P_2 to establish a secret key $K \equiv P_1^{i_2} \equiv P_2^{i_2} \pmod{p}$

Application of discrete logarithm

Example

Diffie-Hellman: Two people 1, 2 establish a secret key K which is n bits long.

- Choose large prime p, longer than n bit
- Choose g such that $g^i \equiv j \pmod{p}$ for some i and all $j \leq p$
- 1 computes $P_1 = g^{i_1} \pmod{n}$ and 2 computes $P_2 = g^{i_2} \pmod{n}$ where i_1, i_2 are their respective SECRETS
- They exchange P_1 and P_2 to establish a secret key $K \equiv P_1^{i_2} \equiv P_2^{i_2} \pmod{p}$

Now, due to the discrete log problem it would be impossible for any interloper determine i_1, i_2 making the key K safe from being cracked

Application of discrete logarithm

Example

Justin and Professor Calculus discussed and chose a prime $p = 11$ with $g = 2$ (also known as primitive root g modulo p).

Application of discrete logarithm

Example

Justin and Professor Calculus discussed and chose a prime $p = 11$ with $g = 2$ (also known as primitive root g modulo p).
Justin chooses $p_1 = 2$, Calculus chooses $p_2 = 4$.

Application of discrete logarithm

Example

Justin and Professor Calculus discussed and chose a prime $p = 11$ with $g = 2$ (also known as primitive root g modulo p).
Justin chooses $p_1 = 2$, Calculus chooses $p_2 = 4$.

$$P_1 \equiv 2^2 \equiv 4 \pmod{11}$$

Application of discrete logarithm

Example

Justin and Professor Calculus discussed and chose a prime $p = 11$ with $g = 2$ (also known as primitive root g modulo p).
Justin chooses $p_1 = 2$, Calculus chooses $p_2 = 4$.

$$P_1 \equiv 2^2 \equiv 4 \pmod{11}$$

$$P_2 \equiv 2^4 \equiv 5 \pmod{11}$$

Application of discrete logarithm

Example

Justin and Professor Calculus discussed and chose a prime $p = 11$ with $g = 2$ (also known as primitive root g modulo p).
Justin chooses $p_1 = 2$, Calculus chooses $p_2 = 4$.

$$P_1 \equiv 2^2 \equiv 4 \pmod{11}$$

$$P_2 \equiv 2^4 \equiv 5 \pmod{11}$$

Calculus gives Justin P_2, Justin gives Calculus P_1

Application of discrete logarithm

Example

Justin and Professor Calculus discussed and chose a prime $p = 11$ with $g = 2$ (also known as primitive root g modulo p).
Justin chooses $p_1 = 2$, Calculus chooses $p_2 = 4$.

$$P_1 \equiv 2^2 \equiv 4 \pmod{11}$$

$$P_2 \equiv 2^4 \equiv 5 \pmod{11}$$

Calculus gives Justin P_2, Justin gives Calculus P_1
Now $K \equiv P_1^4 \equiv P_2^2 \equiv 3 \pmod{11}$

Application of discrete logarithm

Example

Justin and Professor Calculus discussed and chose a prime $p = 11$ with $g = 2$ (also known as primitive root g modulo p).
Justin chooses $p_1 = 2$, Calculus chooses $p_2 = 4$.

$$P_1 \equiv 2^2 \equiv 4 \pmod{11}$$

$$P_2 \equiv 2^4 \equiv 5 \pmod{11}$$

Calculus gives Justin P_2, Justin gives Calculus P_1
Now $K \equiv P_1^4 \equiv P_2^2 \equiv 3 \pmod{11}$
Hence $K = 3$

Discrete Log Problem

Discrete Logarithm Problem

Is there a faster way to find discrete logarithms? That is, if

$$x^{\beta} \equiv a \pmod{n},$$

where x, n, a, β are positive integers, can we find the smallest possible value of y faster than manually checking all exponents up to $\varphi(n) - 1$?

Discrete Logarithm Problem

Question

Are there faster ways than checking every exponent individually?

Discrete Logarithm Problem

Question

Are there faster ways than checking every exponent individually?

Yes!

Discrete Logarithm Problem

Question

Are there faster ways than checking every exponent individually?

Yes! Example on slide 3

Discrete Logarithm Problem

Question

Are there faster ways than checking every exponent individually?

Yes! Some common examples are

1. Baby-step - Giant-step Algorithm: $O(\sqrt{n}\log(n))$
2. Pollard's ρ Algorithm: $O(\sqrt{n})$

but these mainly work when we work in $\mathbb{Z}/p\mathbb{Z}$. What about if we work in another number system?

Discrete Log Problem

[Method] - Solving with Baby-step - Giant-step Algorithm

For solving $x^y \equiv k \pmod{m}$, let $y = u + tv$, $0 \leq u, v \leq t - 1$, where $t = \lfloor m^{1/2} \rfloor + 1$. Hence the problem can be re-expressed as:

$$\begin{aligned} x^{u+tv} &\equiv k \pmod{m} \\ x^{tv} &\equiv k \cdot x^{-u} \pmod{m} \\ (x^t)^v &\equiv k \cdot (x^{-1})^u \pmod{m} \end{aligned}$$

Now if $x^t \pmod{m}$, $x^{-1} \pmod{m}$ can be computed with cost $O(\log m)$, a set $S = \{k \cdot (x^{-1})^u \pmod{m}, \quad u = 0, ..., t-1\}$ can be computed with cost $O(t \log m)$. Sorting S with cost $O(t \log t)$ in increasing order and matching if it equals $(x^t)vc \pmod{m}$ for $v = 0, ..., t-1$ with the same cost, the smallest positive solution can be found. Total cost is $O(,^{1/2} \log m)$